

docker-lab — exemplo completo

Um projeto Python de exemplo, o `job.env` dele e os comandos para rodar, acompanhar, parar e ver o histórico.

Cada arquivo `.env` descreve um job, e um projeto pode ter **quantos arquivos `.env` precisar** (por exemplo `job.env`, `job_treino_grande.env`, `job_analise.env`), todos na raiz do projeto e cada um com um `JOB_NAME` diferente.

1. O projeto de exemplo

estrutura

```
~/meu_projeto/
├─ job.env           ← fica na raiz
├─ venv/             ← Python do projeto
├─ dados/
│   └─ iris.csv
├─ resultados/
├─ requirements.txt ← pacotes do projeto
└─ treino.py         ← script a rodar
```

requirements.txt

```
pandas
scikit-learn
```

criar o venv (uma vez)

```
cd ~/meu_projeto
/usr/bin/python3 -m venv venv
venv/bin/pip install -r requirements.txt
```

Use o caminho completo `/usr/bin/python3`: o `python3` puro está bloqueado no terminal.

2. O job.env

Copie o modelo com `cp /opt/docker_lab/job.env.example ~/meu_projeto/job.env` e deixe assim:

~/meu_projeto/job.env

```
SCRIPT=treino.py           # script a rodar (relativo à pasta do projeto)
PYTHON_BIN=venv/bin/python # Python do venv do projeto
CORES=2                    # quantos cores
MEMORY_GB=4               # RAM máxima (passou disso, o job é encerrado)
GPU=none                  # "none" sem GPU, ou "0" para usar a GPU 0
#GPU_MEMORY_GB=8          # VRAM máxima: obrigatório quando GPU=0
JOB_NAME={USERNAME}_treino # mantenha o {USERNAME}_, troque só o resto
```

3. Rodar

Dentro da pasta do projeto. O job vai para segundo plano, e você pode fechar o terminal.

```
aluno@servidor:~/meu_projeto$ docker-lab run job.env
== Buildando imagem (docker-lab-runner:latest) ==
== Reservando recurso para 'aluno_treino' (cores=2 mem=4GB gpu=none) ==
== Reservado: cores do host = 0,1 ==
== Rodando em segundo plano: /home/aluno/meu_projeto/venv/bin/python treino.py ==
Container: docker-lab-aluno_treino
Log:       /opt/docker_lab/logs/aluno_treino_20261008_103005.log
Ver o log: docker-lab logs job.env
Pra parar: docker-lab stop
```

4. Acompanhar: status e log

```
aluno@servidor:~/meu_projeto$ docker-lab status
Cores: 2 em uso, 10 ainda reserváveis (teto 80% = 12 de 16 totais) – livres de verdade: [2, 3, ..., 15]
Memória: 4GB em uso, 54GB ainda reserváveis (teto 95% = 58GB de 62GB totais)

Jobs ativos:
  aluno_treino: cores=[0, 1] mem=4GB

aluno@servidor:~/meu_projeto$ docker-lab logs job.env
Execuções de 'aluno_treino' (job.env):
  N  INICIO                FIM                  STATUS              SUGESTAO
  1  2026-10-08T10:02:11    2026-10-08T10:02:15  sem RAM             aumente MEMORY_GB (pediu 1GB)
  2  2026-10-08T10:30:05    -                    executando

== Últimas 50 linhas do log da execução 2 ==
n_estimators=50: acurácia=0.953
n_estimators=100: acurácia=0.960

== Log completo da execução 2: /opt/docker_lab/logs/aluno_treino_20261008_103005.log
Ver tudo:      less "/opt/docker_lab/logs/aluno_treino_20261008_103005.log"
Acompanhar ao vivo: tail -f "/opt/docker_lab/logs/aluno_treino_20261008_103005.log"
Ver outra execução: docker-lab logs job.env <N>
```

Cada execução guarda o próprio log. `docker-lab logs job.env` lista todas as execuções desse `.env` e mostra as **últimas 50 linhas** da mais recente. Para ver outra, passe o número: `docker-lab logs job.env 1`. Também aceita o nome do job, como aparece no `docker-lab history`: `docker-lab logs aluno_treino`.

5. Parar

```
aluno@servidor:~/meu_projeto$ docker-lab stop
Jobs ativos:
1) aluno_treino (cores=[0, 1] mem=4GB)
2) Cancelar
Qual você quer parar (número, ou 'Cancelar')? 1
Parando 'docker-lab-aluno_treino'...
Parado. A reserva é liberada sozinha em alguns instantes.
```

Parar **encerra** o job: não é possível retomá-lo depois.

6. Histórico

```
aluno@servidor:~/meu_projeto$ docker-lab history
JOB_NAME  USUARIO  INICIO                FIM                  STATUS              SUGESTAO
-----
aluno_treino  aluno    2026-10-08T10:02:11    2026-10-08T10:02:15  sem RAM             aumente MEMORY_GB (pediu 1GB)
aluno_treino  aluno    2026-10-08T10:30:05    2026-10-08T10:52:19  interrompido
aluno_analise aluno     2026-10-08T11:04:40    2026-10-08T11:31:02  executou
```

executou terminou sem erro

erro o script falhou: veja o motivo com `docker-lab logs`

sem RAM passou do `MEMORY_GB`: aumente no `.env`

sem VRAM passou do `GPU_MEMORY_GB` (`CUDA out of memory`): aumente no `.env` ou reduza o batch size

interrompido foi parado com `docker-lab stop`, ou a máquina reiniciou